An introduction to OData

This article introduces the REST principles to provide a better understanding of the Odata protocol. Some practical examples from CCH Tagetik follow.

Jun 8, 2022 · Knowledge

Title
An introduction to OData

Summary
This article introduces the REST principles to provide a better understanding of the Odata protocol. Some practical examples from CCH Tagetik follow.

From Version

Up to Version

Addressees

Context
This document provides a general introduction to OData without any intention to be considered as an official guide to OData and for which we recommend referring to:

https://www.odata.org/ (https://www.odata.org/)

The same apply to all the existing literature on Restful APIs.

Description

General Overview

OData is a standard protocol for building and consuming RESTful APIs. It may also be viewed as the mean to achieve RESTful APIs.

By definition:

"The Open Data Protocol (OData) enables the creation of REST-based data services, which allow resources, identified using Uniform Resource Locators (URLs) and defined in an Entity Data Model (EDM), to be published and edited by Web clients using simple HTTP messages."

With this regard, an OData resource is any part of the model that can be addressed and so created, read, updated or deleted (e.g. an entity set, entity, property, or operation).
In fact, OData exploits HTTP methods to apply CRUD "activities" on resources (HTTP methods in between parentheses):

Create (POST)
Read (GET)
Update (PUT)
Delete (Delete)

In other words, we could also say that OData is a sort of SQL equivalent for Web.

What does REST mean?

First, REST is not a technology, but it is an architectural style to support API design based on 6 principles.

Before getting into the principles let us imagine an example. A Client-Server communication where the Client asks for Customers and Employees to the Server.

REST principles "say" that an object is created on the server side but only the value of such object (its representation), is sent to the client. What is then actually happening is the transferring of the state of an object. Indeed, REST stands for **Re**presentational **S**tate **T**ransfer, where the "Representational State" is simply the current state of an object.

Also, REST is based on resources, therefore with respect to the above example it would mean that Customer and Employees are both (two) resources.

The information sent to the client are usually provided in XML or JSON format (depending also on the OData version). On a minor side, Json is becoming more popular for web application given that the same data in Json occupy less space on memory (XML Tags makes the file heavier).

REST principles

Rest is based on 6 principles, here below summarized.

1. Uniform Interface

At the end of the day what really matters in RESTful APIs is not where the data are coming from or where they are stored but establishing a common ground (uniform interface) for the communication exchange between client and servers.

A uniform interface should be based on:

- Resources that are uniquely identified via URI.
- Manipulation of resources that happen through representation.
- HTTP methods used to perform CRUD operations.

These 3 constraints define the interface between client and server and by this way client and server can evolve independently (they are therefore decoupled).

2. Stateless

Servers do not contain the client state nor any user session related info. Ironically, after having answered the server does not even know that client exists.
As a major consequence, the specification of the API should provide all the information needed which means assuming that any state from previous calls has been preserved.

For example, it is not possible to make two consecutive queries like the following:

e.g. Where does Mario come from? How old is he?

The second question does not contain all the relevant info because the server would not be able to translate “he” into “Mario”.

- The downside of this approach is that the network traffic might increase.
- On the other side, the server and client are decoupled thus allowing a great scalability of the servers.

3. Client - Server model

The uniform interface separates clients from servers.

4. Caching

Caching is stateful but the requirement for a REST API to be stateless increases traffic network, therefore caching might be used to compensate for some of that. It does not mean that the iteration with the client becomes stateful, but that caching can be used to store highly requested resources in memory to reduce input/output calls to the disk.

5. Layer architecture

Client cannot assume direct connection to server. In most cases client talks to an intermediate software of a hardware system, which means that the backend system can be replaced without affecting the client.

A layered architecture is like the pillow on top of your chair. Someone might change the frame underneath, but nobody would know. In such architecture, each layer only knows about the layer next to it, and no more. This promotes separation-of-concerns. Each layer indeed is responsible for a specific role, it knows how to do its job, and passes information to and from its immediate neighbouring layer to get the job done.

6. Code on demand (optional)

The server sends some code to the client along with the data. At the same time this code tells the client what to do with the data. This way, the client does not need to store business intelligence, but it acts like a simple terminal whose CPU power is used to execute the code.

Example

OData query via browser

In CCH Tagetik it is possible to query the Audit via OData APIs. To do so:

1. To use such APIs it is mandatory to “activate” them via repository (in *System Administration* > *Security* > Checkbox "Enable public REST APIs") and enable the function 0004077 to the role of the “OData” User.
2. Query the Audit using any available browser or ad-hoc interfaces like Postman or Excel.

Audit data can be divided into two parts:

- data management of tables and data
- data processing

An example of the URL to be used for data processing service is the following:

http://server:port/tagetikcpm/api/audit/v1/dataprocessing/ (http://server:port/tagetikcpm/api/audit/v1/dataprocessing/)

which may be translated in this concrete URL example:

http://127.0.0.1:9090/tagetikcpm/api/audit/v1/dataprocessing/ (http://127.0.0.1:9090/tagetikcpm/api/audit/v1/dataprocessing/)

By hitting the above URL, we would receive the list of all application data processing like per the image below:



Data processing list

Further from there, it is possible to ask for a specific data processing by adding the data processing code at the end of the URL, or retrieve the metadata structure of the resource by adding "$metadata" at the end of the URL:

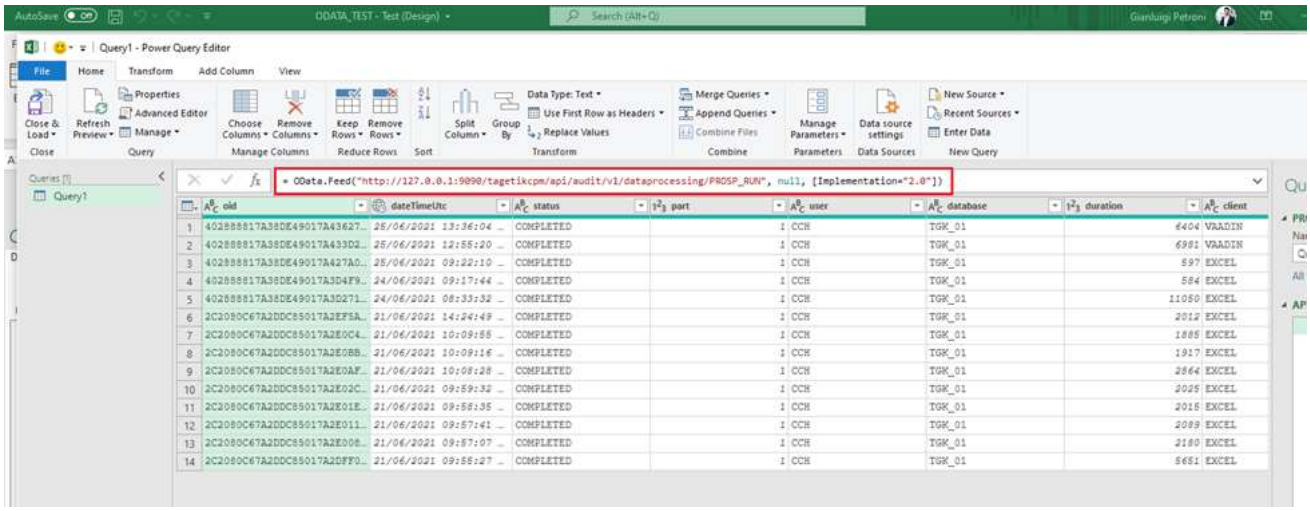
- .../dataprocessing/$metadata
- .../dataprocessing/EXECUTE_QUERY



Data processing code example query

OData query via Excel

The same query done via web could be performed using Excel Odata connection like per the image below:



Excel OData connection (depending on the Excel version, it is available under Data > Get Data > From Other Sources > From OData Feed)

Starting from there a pivot table could be built and refreshed based on the latest Audit data.

oid	dateTimeUtc	status	part	user	database	duration	client	ipClient	aziendaList	categoria	conto	prospetto
402888817A38DE49017A3D271F7E5F16	24/06/2021 08:33	COMPLETED	1	CCH	TGK_01	11050	EXCEL	127.0.0.1				
2C2080C67A2DDC85017A2EF5A70C2BFE	21/06/2021 14:24	COMPLETED	1	CCH	TGK_01	2012	EXCEL	127.0.0.1	EN_00	010	010	E102
2C2080C67A2DDC85017A2E0C47A44AC5	21/06/2021 10:09	COMPLETED	1	CCH	TGK_01	1885	EXCEL	127.0.0.1	EN_00	010	010	E102
2C2080C67A2DDC85017A2E0BB00E42C8	21/06/2021 10:09	COMPLETED	1	CCH	TGK_01	1917	EXCEL	127.0.0.1	EN_00	010	010	E102
2C2080C67A2DDC85017A2E0AF5A03EEA	21/06/2021 10:08	COMPLETED	1	CCH	TGK_01	2864	EXCEL	127.0.0.1	EN_00	010	010	E102
2C2080C67A2DDC85017A2E02C6BB30DA	21/06/2021 09:59	COMPLETED	1	CCH	TGK_01	2025	EXCEL	127.0.0.1	EN_00	010	010	E102
2C2080C67A2DDC85017A2E01E92727E0	21/06/2021 09:58	COMPLETED	1	CCH	TGK_01	2015	EXCEL	127.0.0.1	EN_00	010	010	E102
2C2080C67A2DDC85017A2E0116CA1F8D	21/06/2021 09:57	COMPLETED	1	CCH	TGK_01	2089	EXCEL	127.0.0.1	EN_00	010	010	E102
2C2080C67A2DDC85017A2E008F471CEC	21/06/2021 09:57	COMPLETED	1	CCH	TGK_01	2180	EXCEL	127.0.0.1	EN_00	010	010	E102
2C2080C67A2DDC85017A2DFF08BF1635	21/06/2021 09:55	COMPLETED	1	CCH	TGK_01	5681	EXCEL	127.0.0.1	EN_00	010	010	E102

Count of dateTimeUtc	Column Labels				
	E102	E102 Total	(blank)	(blank) Total	Grand Total
Row Labels	21-Jun		24-Jun		
CCH	9	9	1	1	10
Grand Total	9	9	1	1	10

Excel OData connection: pivot table example

To Learn More

Additional details on Odata are available on the Tagetik help online, the community or on Salesforce.

Was this article helpful? 0 0

Choose a general reason

-- Choose a general reason --

Description

Please, leave us some feedback...

Submit